

CSCI 2540: Advanced Probabilistic Methods in Computer Science

Milan Capoor

Fall 2026

0.1 Sept 10

Why Probabilistic Methods?

- Randomized algorithms (hashing, quicksort, open key cryptography, finance, weather)
- Probabilistic analysis of algorithms (average case analysis, smoothed analysis)
- Modeling data (statistical ML)

Course Topics:

1. Basic randomized algorithms and probabilistic analysis
2. Large deviation bounds (Chernoff and Hoeffding bounds)
3. The probabilistic method
4. Martingale (in discrete space)
5. Sample complexity and statistical learning theory
6. Monte Carlo methods
7. Converge of Monte Carlo Markov Chains

Example (Verifying Matrix Multiplication): Suppose you have three $n \times n$ matrices A, B, C in a boolean field. We want to verify $AB = C$.

A naive matrix multiplication algorithm takes $\Theta(n^3)$ time ($\Theta(n^{2.37})$ at the absolute best). But we would hope verification can be done faster.

Algorithm 1: Freivalds' Randomized algorithm in $\Theta(n^2)$

- 1 Choose a random vector $r \in \{0, 1\}^n$ uniformly at random.
 - 2 Compute Br and $A(Br)$
 - 3 Compute Cr
 - 4 If $A(Br) \neq Cr$, return " $AB \neq C$ ". Otherwise, return " $AB = C$ ".
-

Theorem: If $AB \neq C$ and r is chosen uniformly at random from $\{0, 1\}^n$, then

$$\mathbb{P}(ABr = Cr) \leq \frac{1}{2}$$

Proof:

Take the case $AB \neq C$. Let $D = AB - C \neq 0$. Then

$$ABr = Cr \implies Dr = 0$$

Since $D \neq 0$, there exists some $d_{ij} \in D \neq 0$. By definition of matrix multiplication, we have

$$\sum_{j=1}^n d_{1j} r_j = 0 \iff r_1 = -\frac{1}{d_{11}} \sum_{j=2}^n d_{1j} r_j$$

(assuming WLOG $d_{11} \neq 0$).

Letting $a_1, \dots, a_n$ being the values chosen at random from the set of variables $\{r_1, \dots, r_n\}$, we have

$$\mathbb{P}\left(r_1 = -\frac{1}{d_{11}} \sum_{j=2}^n d_{1j} a_j\right) \leq \frac{1}{2}$$

Hence, if $AB = C$ the algorithm is always correct. If $AB \neq C$ the algorithm is correct with probability at least $1/2$. We can repeat the algorithm k times to reduce the error probability to $1/2^k$.

Probability space: $(\Omega, \mathcal{F}, \mathbb{P})$ for Ω a sample space of outcomes, $\mathcal{F}$ a σ -algebra of allowable events (subsets of Ω), and $\mathbb{P} : \mathcal{F} \rightarrow [0, 1]$ a probability measure.

In a **discrete probability space**, $\omega \in \Omega$ is a simple event and $\mathcal{F} = 2^\Omega$.

Probability function: a function $\mathbb{P} : \mathcal{F} \rightarrow \mathbb{R}$ that satisfies

1. For all $E \subseteq \mathcal{F}$, $0 \leq \mathbb{P}(E) \leq 1$
2. $\mathbb{P}(\Omega) = 1$
3. For any countable collection of pairwise mutually disjoint events $E_1, E_2, \dots \in \mathcal{F}$, we have

$$\mathbb{P}\left(\bigcup_{i \geq 1} E_i\right) = \sum_{i \geq 1} \mathbb{P}(E_i)$$

In a discrete space, each *atomic event* has a finite probability and the probability of an event E is given by the sum of the probabilities of the atomic events in E

Let's return to Freivald's algorithm,

Lemma: Choosing $r = (r_1, \dots, r_n) \in \{0, 1\}^n$ uniformly at random is equivalent to choosing each r_i independently and uniformly at random from $\{0, 1\}$

Hence a more formal version of the earlier proof proceeds:

$$\mathbb{P}(ABr = Cr) = \sum_{x_2, \dots, x_n} \mathbb{P}((ABr = Cr) \cap ((r_2, \dots, r_n) = (x_2, \dots, x_n))) \quad (1)$$

$$\leq \sum_{x_2, \dots, x_n} \mathbb{P}\left(\left[r_1 = -\frac{1}{d_{11}} \sum_{j=2}^n d_{1j} x_j\right] \cap ((r_2, \dots, r_n) = (x_2, \dots, x_n))\right) \quad (2)$$

$$= \sum_{x_2, \dots, x_n} \mathbb{P}\left(r_1 = -\frac{1}{d_{11}} \sum_{j=2}^n d_{1j} x_j\right) \mathbb{P}((r_2, \dots, r_n) = (x_2, \dots, x_n)) \quad (3)$$

$$\leq \sum_{x_2, \dots, x_n} \frac{1}{2} \mathbb{P}((r_2, \dots, r_n) = (x_2, \dots, x_n)) \quad (4)$$

$$= \frac{1}{2} \quad (5)$$

0.2 Sept 15

0.2.1 Deciding Polynomial Identity

Problem: Verify that $P(x) \equiv Q(x)$ for two polynomials P, Q , i.e. $H(x) = P(x) - Q(x) \equiv 0$.

Example: Check if

$$(x+1)(x-2)(x+3)(x-4)(x+5)(x-6) \equiv x^6 - 7x^3 + 25$$

Notational remark: we use $\equiv$ to denote polynomial identity and $=$ for numerical equality.

Algorithm 2: Randomized Polynomial Identity

- 1 Choose $r \in [1, \dots, m]$;
 - 2 If $H(r) = 0$ output “ $P \equiv Q$ ” else output “ $P \not\equiv Q$ ”
-

Theorem: Assume that $H(x) \not\equiv 0$ and $\deg(H) \leq d$. Then if r is chosen uniformly at random from $\{1, \dots, m\}$, we have

$$\mathbb{P}(H(r) = 0) \leq \frac{d}{m}$$

Proof: A polynomial of degree d has at most d roots. Hence, if $H(x) \not\equiv 0$, then there are at most d values of r such that $H(r) = 0$. Since r is chosen uniformly at random from $\{1, \dots, m\}$, we have

$$\mathbb{P}(H(r) = 0) \leq \frac{d}{m}$$

What about multivariate polynomials?

Example:

$$H(x, y, z, w) = x^5 y^3 (z^3 - w^4) + x^3 (z^5 - w^3) (x^2 y^4 - w^2) \stackrel{?}{=} 0$$

Define the degree of a multivariate polynomial $H(x_1, \dots, x_n)$ as the maximum sum of the degrees in each of its monomials. For example, the degree of $H(x, y, z, w)$ above is $3 + 5 + 2 + 4 = 14$.

For this setting, no efficient deterministic algorithm is known. A polynomial with n variables and degree d can have up to $\binom{n+d}{d}$ monomials, which is exponential in n

Theorem (Schwartz-Zippel): Assume that $H(x_1, \dots, x_n) \not\equiv 0$ and $\deg(H) \leq d$. Then if $r_1, \dots, r_n$ are chosen independently and uniformly at random from a set S (equivalently, sampling $(r_1, \dots, r_n)$ from S^n), then

$$\mathbb{P}(H(r_1, \dots, r_n) = 0) \leq \frac{d}{|S|}$$

Proof: By induction on n .

For $n = 1$, H has at most d roots and by the theorem above,

$$\mathbb{P}(H(r_1) = 0) \leq \frac{d}{|S|}$$

For $n > 1$, H can be written as a polynomial in x_n with coefficients that are polynomials in $x_1, \dots, x_{n-1}$:

$$H(x_1, \dots, x_n) = \sum_{i \geq 0} x_n^i f_i(x_1, \dots, x_{n-1})$$

Let $k \leq d$ be the largest index for which $f_k \not\equiv 0$. Then $\deg(f_k) \leq d - k$ and by the induction hypothesis,

$$\mathbb{P}(f_k(r_1, \dots, r_{n-1}) = 0) \leq \frac{d - k}{|S|}$$

If $f_k(r_1, \dots, r_{n-1}) \neq 0$, then $H(r_1, \dots, r_{n-1}, x_n)$ is a nonzero univariate polynomial in x_n of degree k . Hence by the univariate case,

$$\mathbb{P}[H(r_1, \dots, r_n) = 0 \mid f_k(r_1, \dots, r_{n-1}) \neq 0] \leq \frac{k}{|S|}$$

Then since

$$\begin{aligned}\mathbb{P}(A) &= \mathbb{P}(A \cap B) + \mathbb{P}(A \cap B^c) \\ &= \mathbb{P}(A \mid B)\mathbb{P}(B) + \mathbb{P}(A \mid B^c)\mathbb{P}(B^c) \\ &\leq \mathbb{P}(A \mid B) + \mathbb{P}(B^c)\end{aligned}$$

we have

$$\begin{aligned}\mathbb{P}(H(r_1, \dots, r_n) = 0) &= \mathbb{P}[H(r_1, \dots, r_n) = 0 \mid f_k(r_1, \dots, r_{n-1} \neq 0)] + \mathbb{P}[f_k(r_1, \dots, r_{n-1}) = 0] \\ &\leq \frac{k}{|S|} + \frac{d-k}{|S|} \\ &= \frac{d}{|S|}\end{aligned}$$

Recall the definition of conditional probability.

Conditional Probability:

$$\mathbb{P}(E_1 \mid E_2) = \frac{\mathbb{P}(E_1 \cap E_2)}{\mathbb{P}(E_2)}$$

where we can think of conditioning on E_2 as restricting the sample space to E_2 . Hence, we have

$$\mathbb{P}(E_n) = \sum_{i=1}^n \mathbb{P}(E_n \mid E_i) \mathbb{P}(E_i)$$

This exposes a general method for the course: “Suppose some nice condition holds and analyze the algorithm conditioned on that event. Then bound the probability of that event not holding.”

This theorem has a **one-sided error**: if $H \equiv 0$, the algorithm is always correct. If $H \not\equiv 0$, the algorithm is correct with probability at least $1 - d/|S|$. We can repeat the algorithm ℓ independent times to reduce the error probability to $(d/|S|)^\ell$.

Independent Events: $E \perp F$ iff

$$\mathbb{P}(E \cap F) = \mathbb{P}(E) \cdot \mathbb{P}(F)$$

More generally, events $E_1, \dots, E_n$ are **mutually independent** iff for any subset $I \subseteq \{1, \dots, n\}$,

$$\mathbb{P}\left(\bigcap_{i \in I} E_i\right) = \prod_{i \in I} \mathbb{P}(E_i)$$

In particular, if $E \perp F$, then $\mathbb{P}(E \mid F) = \mathbb{P}(E)$.

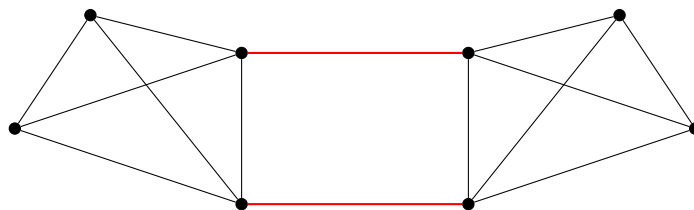
0.3 Sept 17

Min-cut set: a minimum set of edges that disconnects a graph.

Trivially, we can iterate over all nodes and cut all edges for the node with the minimum degree. This gives an upper bound on the size of the min-cut:

$$\text{min-cut} \leq \min_{v \in V} \deg(v)$$

Consider, however, this graph:



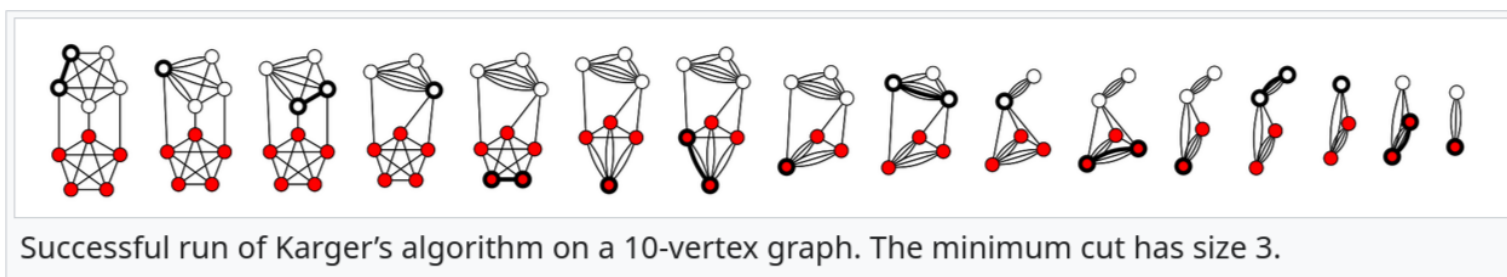
Every node has degree 4, but the min-cut is 2.

As we will see, this is a deep problem with many applications. This problem happens to have a deterministic polynomial time algorithm related to max-flow but it is complicated and involves an $O(|V|^2)$ algorithm run between every pair of nodes ($O(\binom{|V|}{2})$ pairs).

Algorithm 3: Randomized Min-cut Algorithm

```

1 for  $i=1:n-2$  do
2   | Pick an edge uniformly at random;
3   | Contract the two vertices connected by that edge, eliminate all edges connecting the two vertices;
4 Output the set of edges connecting the two remaining vertices;
  
```



Notice that this algorithm fails precisely when we contract an edge in the min-cut. Following the strategy we saw last time, we would like to find the condition that makes the algorithm fail deterministically and then bound the probability of that condition holding.

Theorem:

1. The algorithm outputs a min-cut edge-set with probability

$$\geq \frac{2}{n(n-1)}$$

2. The smallest set output in $O(n^2 \log n)$ iterations of the algorithm gives a correct answer with probability $1 - 1/n^2$.

Proof: Assume the graph has a min-cut set of k edges. We compute the probability of finding one particular such set C .

Trivially,

$$\mathbb{P}(\text{alg. returns any min-cut}) \geq \mathbb{P}(\text{alg. returns min-cut } C)$$

Lemma: If no edge of C was contracted, the algorithm outputs C .

First, notice that if no edge of C was contracted, no edge of C was eliminated. To see this, let X and Y be the two sets of vertices connected by edges in C . If no edge of C was contracted, then no vertex in X was contracted with a vertex in Y . Hence, all edges in C remain in the graph and are output by the algorithm.

As a corollary, if the algorithm terminates before contracting any edge in C , it gives the correct answer.

Finally, to show the one-sided error property, notice that vertex contraction does not reduce the size of the min-cut set but does reduce the size of the original graph. Intuitively, contracting an edge *reduces the size of the sample space*

Lemma:

$$\mathbb{P}(\text{no edge in } C \text{ is contracted}) \geq \frac{2}{n(n-1)}$$

Let E_i be the event “no edge contracted in iteration i is in C ” so

$$F_i = \bigcap_{j=1}^i E_j = \text{“no edge of } C \text{ is contracted in the first } i \text{ iterations”}$$

Since the min-cut set has k edges, all vertices have degree $\geq k$ (else the min-cut set would be smaller). Hence, the number of edges in the graph $\geq nk/2$.

$$\mathbb{P}(E_1) = \mathbb{P}(F_1) = 1 - \frac{2k}{nk} = 1 - \frac{2}{n}$$

Conditioning on E_1 , after the first contraction, we have $n-1$ vertices and $k(n-1)/2$ edges but still k edges in the min-cut set. By analogous argument,

$$\begin{aligned} \mathbb{P}(E_2) &= \mathbb{P}(E_2 \mid E_1) \cdot \mathbb{P}(E_1) + \mathbb{P}(E_2 \mid \overline{E_1}) \cdot \mathbb{P}(\overline{E_1}) \\ \mathbb{P}(E_2 \mid E_1) &\geq 1 - \frac{k}{|G_1|} \quad (\text{where } |G_1| = \# \text{ edges after iter 1}) \\ &\geq 1 - \frac{k}{(n-1)k/2} \\ &= 1 - \frac{2}{n-1} \end{aligned}$$

Of course, since $\mathbb{P}(E_1) = \mathbb{P}(F_1)$,

$$\mathbb{P}(E_2 \mid F_1) = \mathbb{P}(E_2 \mid E_1) \geq 1 - \frac{k}{k(n-1)/2} \geq 1 - \frac{2}{n-1}$$

Inductively,

$$\begin{aligned} \mathbb{P}(E_i \mid F_{i-1}) &= \mathbb{P}\left(E_i \mid \bigcap_{j=1}^{i-1} E_j\right) \\ &\geq 1 - \frac{k}{k(n-i+1)/2} \\ &= 1 - \frac{2}{n-i+1} \end{aligned}$$

How do we compute F_{n-1} given a sequence of $\mathbb{P}(E_i \mid F_{i-1})$? By chain rule:

$$\begin{aligned} \mathbb{P}(F_n) &= \mathbb{P}\left(\bigcap_{j=1}^n E_j\right) \\ &= \mathbb{P}(E_{n-1} \mid F_{n-1}) \cdot \mathbb{P}(F_{n-1}) \\ &= \mathbb{P}(E_n \mid F_{n-1}) \cdot \mathbb{P}(E_{n-1} \mid F_{n-2}) \cdots \mathbb{P}(E_2 \mid F_1) \cdot \mathbb{P}(E_1) \end{aligned}$$

We are interested in $\mathbb{P}(F_{n-2})$ since the algorithm terminates after $n - 2$ iterations. Simply plugging in,

$$\begin{aligned}
 \mathbb{P}(F_{n-2}) &= \mathbb{P}(F_1) \cdot \prod_{j=2}^{n-2} \mathbb{P}(E_j \mid F_{j-1}) \\
 &\geq \prod_{i=1}^{n-2} \left(1 - \frac{2}{n-i+1}\right) \\
 &= \prod_{i=1}^{n-2} \frac{n-i-1}{n-i+1} \\
 &= \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdots \frac{2}{n-1} \\
 &= \frac{2}{n(n-1)} \quad \blacksquare
 \end{aligned}$$

Proof of Part 2: Assume that we run the algorithm $n(n-1) \log n$ times and output the minimum cut set found across the iterations. We claim the probability of not finding a min-cut set is at most $1/n^2$.

From the lemma proving the one-sided error, the output cannot be smaller than the min-cut set. Hence, the probability that C is *not* the output of any of the $n(n-1) \log n$ independent iterations is

$$\leq \left(1 - \frac{2}{n(n-1)}\right)^{n(n-1) \log n} \leq e^{-2 \log n} = \frac{1}{n^2}$$

Via Taylor Series,

$$e^{-x} = 1 - x + \frac{x^2}{2!} - \frac{x^3}{3!} + \cdots$$

So for $x < 1$,

$$1 - x \leq e^{-x}$$

Once again, we can repeat this algorithm to reduce the error because it has **one-sided error** (the algorithm is completely correct for one specific outcome while bounding the error probability for the other outcome)

0.4 Sept 22

Algorithm 4: Quicksort

- 1 **Input:** An array S
- 2 **Output:** The array S in sorted order.
- 3 Choose an element $y \in S$
- 4 Compare all elements of S to y . Let

$$\begin{aligned}
 S_1 &= \{x \in S - \{y\} \mid x \leq y\} \\
 S_2 &= \{x \in S - \{y\} \mid x > y\}
 \end{aligned}$$

- 5 Return $\text{Quicksort}(S_1) \cup \{y\} \cup \text{Quicksort}(S_2)$
-

What can we say about the run time of this algorithm? Well, it depends on what it means to “choose an element $y \in S$ ”.

In the *randomized* version of the algorithm, we choose y uniformly at random from S .

But if we were to choose y deterministically (say splitting the array in half), we could end up with a worst case run time of $O(n^2)$ or else do well and get $O(n \log n)$ run time.

Let $T(n)$ be the number of comparisons made by the algorithm on an array of size n .

Here, $T(n)$ is a *random variable*.

Random Variable: X is a random variable if it is a function $X : \Omega \rightarrow \mathbb{R}$ from the sample space Ω to the real numbers.

Random Vector: $X^d : \Omega \rightarrow \mathbb{R}^d$

Discrete random variable: a random variable that takes on only a finite or countably infinite number of values. In this case, we use notation

$$\mathbb{P}(X = a) = \mathbb{P}(\{s \in \Omega : X(s) = a\}) = \sum_{s \in \Omega : X(s) = a} \mathbb{P}(s)$$

Viewing random variables as functions of events, we can generalize the definition of independence to random variables:

Independent Random Variables: $X \perp Y$ iff for all $x, y \in \Omega$,

$$\mathbb{P}(X = x \wedge Y = y) = \mathbb{P}(X = x) \cdot \mathbb{P}(Y = y)$$

Similarly, $X_1, \dots, X_n$ are **mutually independent** iff for all $a_1, \dots, a_n \in \mathbb{R}$,

$$\mathbb{P}(X_1 = a_1 \wedge \dots \wedge X_n = a_n) = \prod_{i=1}^n \mathbb{P}(X_i = a_i)$$

Median: the median(s) of a random variable is a value m such that

$$\begin{cases} \mathbb{P}(x < m) \leq \frac{1}{2} \\ \mathbb{P}(X > m) < \frac{1}{2} \end{cases}$$

for a symmetric distribution, the median is the same as the mean.

Finally, we can define the expectation of a random variable:

$$\mathbb{E}[X] = \sum_{i \in \text{range}(X)} i \cdot \mathbb{P}(X = i)$$

where $\mathbb{E}[X]$ exists iff $\sum_i |i| \mathbb{P}(X = i) < \infty$ (i.e. the sum converges uniformly).

Theorem: The expected number of steps in the randomized version of quicksort is

$$\mathbb{E}[T(n)] = O(n \log n)$$

Proof: Let $s_1, \dots, s_n$ be the sorted elements of S . For $i = 1 : n$ and $j > i$, define

$$X_{i,j} = \begin{cases} 1 & \text{if } s_i, s_j \text{ are compared in the algorithm} \\ 0 & \text{otherwise} \end{cases}$$

(for example, after the first partition elements of S_1 are never compared to elements of S_2). Then

$$T(n) = \sum_{i=1}^n \sum_{j>i} X_{i,j}$$

By linearity of expectation,

$$\mathbb{E}[T(n)] = \sum_{i=1}^n \sum_{j>i} \mathbb{E}[X_{i,j}]$$

Since $X_{i,j}$ is a Bernoulli random variable,

$$\mathbb{E}[X_{i,j}] = 0 \cdot \mathbb{P}(X_{i,j} = 0) + 1 \cdot \mathbb{P}(X_{i,j} = 1) = \mathbb{P}(X_{i,j} = 1)$$

Notice that s_i and s_j are compared if and only if one of them is chosen as the pivot before any of the $j-i-1$ elements between them is chosen.

Since elements are chosen uniformly at random, $X_{i,j} = 1$ iff the first pivot chosen in the set is $\{s_i, s_{i+1}, \dots, s_j\}$ is either s_i or s_j .

$$\mathbb{E}[X_{i,j}] = \frac{2}{j-i+1}$$

Substituting back into the expectation of $T(n)$, we have

$$\begin{aligned} \mathbb{E}[T(n)] &= \sum_{i=1}^n \sum_{j>i} \frac{2}{j-i+1} \\ &\leq n \sum_{k=1}^n \frac{2}{k} \\ &\leq 2nH_n \\ &= 2n \log n + O(n) \end{aligned}$$

where H_n is the harmonic series

$$H_n = \sum_{i=1}^n \frac{1}{i} \approx \int_1^n \frac{1}{x} dx = \log n$$

Notice in particular that this analysis did not require any knowledge of the distribution of the input array S .

Exercise: Prove

1. $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$
2. for $c \in \mathbb{R}$ and X an RV, $\mathbb{E}[cX] = c\mathbb{E}[X]$

Exercise: Prove $\log n \leq H_n \leq \log n + 1$

Algorithm 5: Deterministic Quicksort

- 1 **Input:** An array S
- 2 **Output:** The array S in sorted order.
- 3 Let y be the first element in S
- 4 Compare all elements of S to y . Let

$$S_1 = \{x \in S - \{y\} \mid x \leq y\}$$

$$S_2 = \{x \in S - \{y\} \mid x > y\}$$

- 5 Return $\text{DetQuicksort}(S_1) \cup \{y\} \cup \text{DetQuicksort}(S_2)$
-

Theorem: the expected run time of deterministic Quicksort on a random permutation of S is $O(n \log n)$

Proof: Set $X_{i,j}$ as before. If all permutations have equal probability, then permutations of $S_i, \dots, S_j$ have equal probability.

Again,

$$\mathbb{P}(X_{i,j} = 1) = \frac{2}{j - i + 1}$$

and

$$\mathbb{E}[T(n)] = \sum_{i=1}^n \sum_{j>i} \frac{2}{j - i + 1} = O(n \log n)$$

This opens the way for two classifications of uses of randomness in algorithms:

1. **Randomized algorithm:** algorithms where the analysis is true for all inputs, repeated runs are independent, and the sample space is the space of random choices made by the algorithm.
2. **Probabilistic Analysis of algorithms:** The sample space is the space of all possible inputs; If the algorithm is deterministic repeated runs give the same output.

In particular we will be interested in:

1. **Monte Carlo Algorithm:** a randomized algorithm that may produce an incorrect solution.
2. **Las Vegas algorithm:** a randomized algorithm that always produces the correct output.

For decision problems: A *one-side error Monte Carlo algorithm* errs only one one possible output, otherwise it is a two-side error algorithm.

In both types of algorithms the run-time is a random variable.

The Random Allocations Paradigm: set of algorithms where we randomly allocate m balls into n bins.

- Used for load balancing, hashing, routing, distributed computing, and more

Algorithm 6: The Coupon Collector Problem

- 1 **Input:** n bins
 - 2 **Output:** The number of balls placed until every bin has at least one ball.
 - 3 Place balls independently and uniformly at random into n bins.
 - 4 Continue until every bin has at least one ball. Let X be the number of balls placed.
-

Theorem:

1. $\mathbb{E}[X] = nH_n = n \log n + \Theta(n)$
2. $\mathbb{P}(X \geq 2H_n) = \Theta(1/n)$

Let X_i be the number of balls placed when there were $i - 1$ non-empty bins. Then

$$X = \sum_{i=1}^n X_i$$

so X_i is the number of trials until we hit an empty box, that is: X_i is a geometric random variable with success probability

$$p_i = 1 - \frac{i-1}{n}$$

Proof:

Geometric random variable: X is a geometric random variable with success probability p on $n = 1, 2, \dots$

$$\mathbb{P}(X = n) = (1 - p)^{n-1}p$$

(Commonly used for the number of trials until the first success in a sequence of independent Bernoulli trials.)

Lemma (Memoryless Distribution): For a geometric random variable X with success probability p ,

$$\mathbb{P}(X = n + k \mid X > k) = \mathbb{P}(X = n)$$

Proof:

$$\begin{aligned}
 \mathbb{P}(X = n + k \mid X > k) &= \frac{\mathbb{P}(X = n + k \wedge X > k)}{\mathbb{P}(X > k)} \\
 &= \frac{\mathbb{P}(X = n + k)}{\mathbb{P}(X > k)} \\
 &= \frac{(1 - p)^{n+k-1}p}{\sum_{i=k}^{\infty} (1 - p)^i p} \\
 &= \frac{(1 - p)^{n+k-1}p}{(1 - p)^k} \\
 &= (1 - p)^{n-1}p \\
 &= \mathbb{P}(X = n)
 \end{aligned}$$

Note that the geometric distribution is the only discrete memoryless distribution (and the exponential distribution is the only continuous memoryless distribution).

Exercise: Prove that the expectation of $X \sim G(p)$ is

$$\mathbb{E}[X] = \sum_{i \geq 1} i(1 - p)^{i-1}p = \frac{1}{p}$$

Conditional Expectations:

$$\mathbb{E}[Y \mid Z = z] = \sum_y yP(Y = y \mid Z = z)$$

Lemma: For any random variables X, Y

$$\mathbb{E}[X] = \mathbb{E}_Y[E_X[X | Y]] = \sum_y \mathbb{P}(Y = y) \mathbb{E}[X | Y = y]$$

Proof:

$$\begin{aligned} \mathbb{E}_Y[E_X[X | Y]] &= \sum_y \mathbb{P}(Y = y) \mathbb{E}[X | Y = y] \\ &= \sum_y \mathbb{P}(Y = y) \sum_x x P(X = x | Y = y) \\ &= \sum_y \mathbb{P}(Y = y) \sum_x x \frac{\mathbb{P}(X = x, Y = y)}{\mathbb{P}(Y = y)} \\ &= \sum_y \sum_x x P(X = x, Y = y) \\ &= \sum_x x P(X = x) \\ &= E[X] \end{aligned}$$

This gives us an easier way to compute the expectation of $X \sim G(p)$:

Proposition: Let $X \sim G(p)$ be a geometric random variable with success probability p . Then

$$\mathbb{E}[X] = \frac{1}{p}$$

Proof:

Let $Y = 1$ if the first trial is a success and $Y = 0$ otherwise. Then

$$\begin{aligned} \mathbb{E}[X] &= \mathbb{P}(Y = 0) \mathbb{E}[X | Y = 0] + \mathbb{P}(Y = 1) \mathbb{E}[X | Y = 1] \\ &= (1 - p) \mathbb{E}[X | Y = 0] + p \mathbb{E}[X | Y = 1] \end{aligned}$$

Trivially, $\mathbb{E}[X | Y = 1] = 1$. In the case $Y = 0$, let Z be the number of trials until the first success after the first trial.

By the memoryless property, $Z \sim G(p)$ and $\mathbb{E}[Z] = \mathbb{E}[X]$. Hence,

$$\mathbb{E}[X] = (1 - p) \mathbb{E}[Z + 1] + p = (1 - p) \mathbb{E}[Z] + 1 \implies \mathbb{E}[X] = \frac{1}{p}$$